Nama : Yoga Adi Nugraha

NIM : H1D024118

Matkul : Praktikum Kecerdasan Buatan

Shift : D

PERTEMUAN 4 SISTEM PAKAR

Source Code

```python
import tkinter as tk
from tkinter import messagebox
```

Program diawali dengan proses impor pustaka GUI menggunakan tkinter. Pustaka tkinter diimpor dengan alias tk untuk mempermudah pemanggilan komponen antarmuka seperti jendela, label, tombol, dan frame. Selain itu, modul messagebox diimpor secara khusus untuk menampilkan dialog informasi dan peringatan kepada pengguna. Pada aplikasi sistem pakar ini, messagebox digunakan untuk menampilkan hasil diagnosa ketika proses tanya jawab gejala selesai. Dengan demikian, dua baris impor tersebut merupakan fondasi utama agar aplikasi dapat berjalan secara interaktif berbasis antarmuka grafis.

```python
# 1. KNOWLEDGE BASE (Basis Pengetahuan)
# Struktur: "Nama Masalah": ["Gejala1", "Gejala2", ...]
basis_pengetahuan = {
    "Kerusakan RAM": ["sering_bluescreen", "bunyi_beep_berulang", "sering_hang"],
    "Masalah Overheat (Prosesor)": ["laptop_tiba_tiba_mati", "kipas_berisik", "suhu_bawah_panas"],
    "Kerusakan Harddisk/SSD": ["booting_sangat_lama", "muncul_pesan_disk_error", "file_sering_corrupt"],
    "Baterai Bocor/Rusak": ["cepat_habis_daya", "hanya_nyala_saat_dicharge", "indikator_baterai_silang"],
    "Kerusakan Keyboard": ["tombol_tertekan_sendiri", "beberapa_tombol_mati", "bunyi_beep_saat_ngetik"]
}
```

pengetahuan pakar direpresentasikan dalam bentuk aturan berbasis gejala. Setiap jenis kerusakan, seperti Kerusakan RAM, Masalah Overheat (Prosesor), Kerusakan Harddisk/SSD, Baterai Bocor/Rusak, dan Kerusakan Keyboard, memiliki tiga gejala khas yang disimpan dalam list. Representasi ini memudahkan proses inferensi karena program cukup memeriksa

apakah gejala yang dipilih pengguna sesuai dengan gejala pada aturan tertentu. Jika seluruh gejala pada satu aturan terpenuhi, maka sistem menyimpulkan bahwa kerusakan tersebut kemungkinan terjadi. Dengan demikian, basis_pengetahuan menjadi komponen inti yang menghubungkan input gejala pengguna dengan hasil diagnosa sistem.

```python
# 2. DAFTAR PERTANYAAN GEJALA
bank_pertanyaan = [
    ("sering_bluescreen", "Apakah laptop sering muncul Blue Screen (BSOD)?"),
    ("bunyi_beep_berulang", "Apakah terdengar bunyi 'beep' saat laptop dinyalakan?"),
    ("sering_hang", "Apakah sistem sering membeku (hang) secara tiba-tiba?"),
    ("laptop_tiba_tiba_mati", "Apakah laptop sering mati mendadak saat digunakan?"),
    ("kipas_berisik", "Apakah suara kipas terdengar sangat kencang/berisik?"),
    ("suhu_bawah_panas", "Apakah bagian bawah laptop terasa sangat panas?"),
    ("booting_sangat_lama", "Apakah proses masuk ke Windows/OS sangat lambat?"),
    ("muncul_pesan_disk_error", "Apakah sering muncul pesan 'Disk Error' atau 'No Bootable Device'?"),
    ("file_sering_corrupt", "Apakah file yang disimpan sering rusak atau tidak bisa dibuka?"),
    ("cepat_habis_daya", "Apakah persentase baterai berkurang sangat cepat?"),
    ("hanya_nyala_saat_dicharge", "Apakah laptop mati jika charger dicabut?"),
    ("indikator_baterai_silang", "Apakah ada tanda silang merah pada ikon baterai?"),
    ("tombol_tertekan_sendiri", "Apakah ada karakter yang muncul sendiri saat mengetik?"),
    ("beberapa_tombol_mati", "Apakah ada beberapa tombol keyboard yang tidak berfungsi?"),
    ("bunyi_beep_saat_ngetik", "Apakah terdengar bunyi beep saat menekan tombol tertentu?")
]
```

Pada bagian ini, program mendefinisikan variabel bank_pertanyaan yang berisi daftar pertanyaan terkait gejala kerusakan laptop. Struktur data yang digunakan adalah list berisi tuple, dengan format (kode_gejala, kalimat_pertanyaan). Kode gejala seperti sering_bluescreen atau kipas_berisik berfungsi sebagai identitas gejala yang nantinya dicocokkan dengan basis pengetahuan, sedangkan kalimat pertanyaan ditampilkan langsung ke pengguna melalui antarmuka GUI. Dengan pemisahan ini, sistem menjadi lebih terstruktur karena logika diagnosis menggunakan kode gejala, sementara interaksi pengguna tetap menggunakan bahasa yang mudah dipahami. Selain itu, urutan pertanyaan dalam bank_pertanyaan menentukan alur sesi tanya jawab dari awal sampai akhir diagnosa.

```python
class SistemPakarLaptop:
    def __init__(self, root):
        self.root = root
        self.root.title("Expert System - PC Troubleshooting")
        self.root.geometry("500x350")

        self.gejala_user = []
        self.nomor_pertanyaan = 0

        # UI Elements
        self.judul_aplikasi = tk.Label(root, text="DIAGNOSA KERUSAKAN LAPTOP", font=("Arial", 14, "bold"))
        self.judul_aplikasi.pack(pady=10)

        self.teks_pertanyaan = tk.Label(root, text="Klik 'Mulai' untuk cek kerusakan", font=("Arial", 11), wraplength=400)
        self.teks_pertanyaan.pack(pady=30)

        self.tombol_mulai = tk.Button(root, text="Mulai Diagnosa", font=("Arial", 10, "bold"),
                        bg="#2ecc71", fg="white", pady=10, padx=20, command=self.mulai_tanya)
        self.tombol_mulai.pack()

        # Frame Tombol Jawaban
        self.kotak_jawaban = tk.Frame(root)
        self.tombol_ya = tk.Button(self.kotak_jawaban, text="YA", width=12, bg="#3498db", fg="white", command=lambda: self.jawab('y'))
        self.tombol_tidak = tk.Button(self.kotak_jawaban, text="TIDAK", width=12, bg="#e74c3c", fg="white", command=lambda: self.jawab('t'))

        self.tombol_ya.pack(side=tk.LEFT, padx=20)
        self.tombol_tidak.pack(side=tk.LEFT, padx=20)

    def mulai_tanya(self):
        self.gejala_user = []
        self.nomor_pertanyaan = 0
        self.tombol_mulai.pack_forget()
        self.kotak_jawaban.pack(pady=20)
        self.tampilkan_pertanyaan()

    def tampilkan_pertanyaan(self):
        if self.nomor_pertanyaan < len(bank_pertanyaan):
            _, kalimat_tanya = bank_pertanyaan[self.nomor_pertanyaan]
            self.teks_pertanyaan.config(text=kalimat_tanya)
        else:
            self.proses_hasil()

    def jawab(self, respon):
        if respon == 'y':
            kode_gejala = bank_pertanyaan[self.nomor_pertanyaan][0]
            self.gejala_user.append(kode_gejala)

        self.nomor_pertanyaan += 1
        self.tampilkan_pertanyaan()

    def proses_hasil(self):
        daftar_hasil = []
        for nama_kerusakan, syarat_gejala in basis_pengetahuan.items():
            # Jika semua gejala syarat terpenuhi
            if all(g in self.gejala_user for g in syarat_gejala):
                daftar_hasil.append(nama_kerusakan)

        self.kotak_jawaban.pack_forget()

        if daftar_hasil:
            kesimpulan = "\n".join([f"- {item}" for item in daftar_hasil])
            messagebox.showinfo("Hasil Diagnosa", f"Berdasarkan gejala, kemungkinan besar terjadi:\n\n{kesimpulan}")
        else:
            messagebox.showwarning("Hasil Diagnosa", "Gejala tidak mencukupi untuk mendiagnosa kerusakan spesifik.")

        self.tombol_mulai.config(text="Cek Lagi")
        self.tombol_mulai.pack(pady=10)
        self.teks_pertanyaan.config(text="Selesai! Ingin melakukan diagnosa ulang?")

if __name__ == "__main__":
    jendela = tk.Tk()
    app = SistemPakarLaptop(jendela)
    jendela.mainloop()
```

Bagian class pada Tugas.py adalah inti aplikasi karena mengatur alur tanya jawab gejala, proses inferensi, dan tampilan hasil diagnosa.

Kelas SistemPakarLaptop merepresentasikan implementasi sistem pakar berbasis GUI menggunakan tkinter. Kelas ini menerima objek root sebagai jendela utama, kemudian mengelola seluruh proses diagnosis dari tahap awal hingga penampilan kesimpulan. Pada method inisialisasi Tugas.py, program menetapkan judul dan ukuran jendela, menyiapkan variabel gejala_user untuk menyimpan gejala yang dijawab “ya”, serta nomor_pertanyaan

sebagai penanda urutan pertanyaan. Selain itu, dibuat komponen antarmuka berupa label judul, teks pertanyaan, tombol mulai, dan tombol jawaban YA/TIDAK.

Method mulai_tanya pada Tugas.py berfungsi mereset sesi diagnosis. Saat tombol “Mulai Diagnosa” ditekan, data gejala sebelumnya dikosongkan, indeks pertanyaan dikembalikan ke awal, tombol mulai disembunyikan, lalu sistem menampilkan pertanyaan pertama.

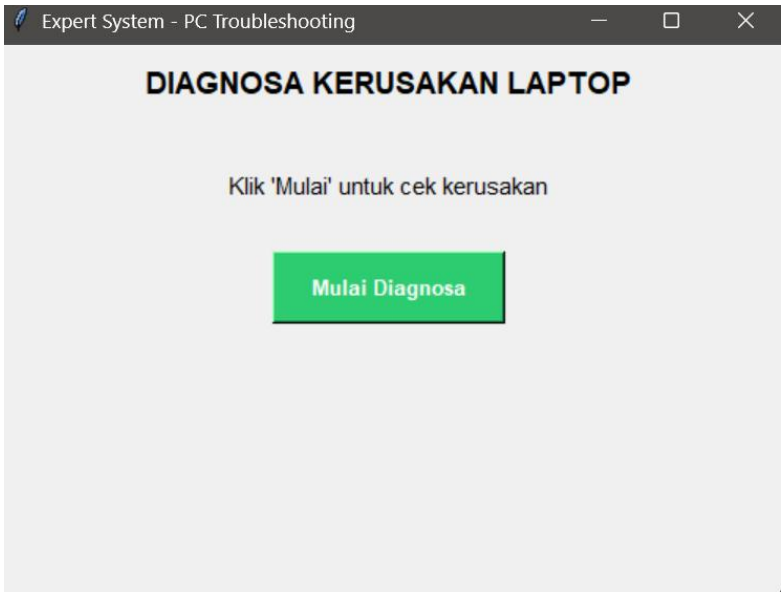
Method tampilkan_pertanyaan pada Tugas.py mengatur perpindahan pertanyaan. Jika indeks pertanyaan masih dalam batas bank_pertanyaan, teks pertanyaan diperbarui sesuai urutan. Jika seluruh pertanyaan sudah selesai, sistem langsung memanggil proses_hasil.

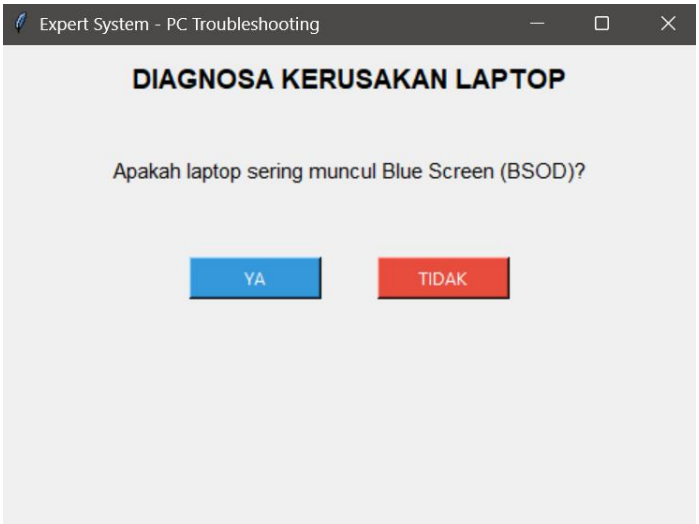
Method jawab pada Tugas.py menangani input pengguna. Jika pengguna memilih “YA”, kode gejala pada pertanyaan aktif disimpan ke gejala_user. Setelah itu indeks pertanyaan dinaikkan dan program menampilkan pertanyaan berikutnya.

Method proses_hasil pada Tugas.py menjalankan inferensi berbasis aturan. Program memeriksa setiap kerusakan dalam basis_pengetahuan dan menggunakan logika all() untuk memastikan semua gejala syarat ada pada gejala_user. Jika cocok, kerusakan dimasukkan ke daftar_hasil. Hasil akhir ditampilkan melalui messagebox: showinfo jika ada diagnosis yang cocok, atau showwarning jika gejala belum cukup. Setelah itu tombol diubah menjadi “Cek Lagi” agar pengguna dapat mengulang diagnosis.

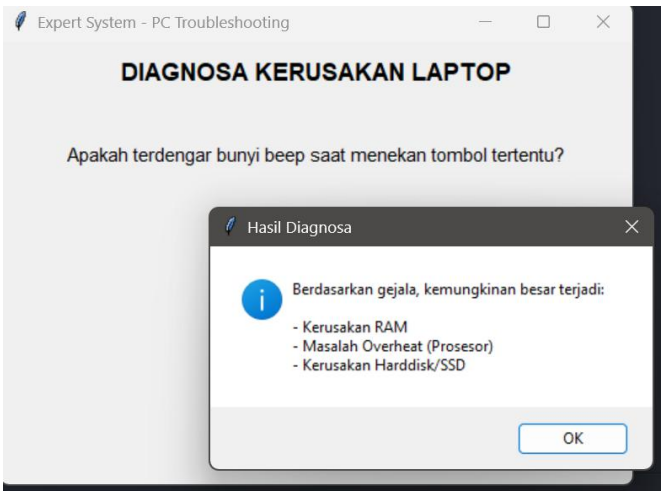
Bagian eksekusi utama pada Tugas.py menjalankan aplikasi dengan membuat objek Tk, menginisialisasi class SistemPakarLaptop, lalu memulai event loop menggunakan mainloop().

Contoh GUI




Expert System - PC Troubleshooting
DIAGNOSA KERUSAKAN LAPTOP
Apakah laptop sering muncul Blue Screen (BSOD)?
YA
TIDAK

Expert System - PC Troubleshooting
DIAGNOSA KERUSAKAN LAPTOP
Apakah sering muncul pesan 'Disk Error' atau 'No Bootable Device'?
YA
TIDAK


Expert System - PC Troubleshooting
DIAGNOSA KERUSAKAN LAPTOP
Apakah terdengar bunyi beep saat menekan tombol tertentu?
Hasil Diagnosa
Berdasarkan gejala, kemungkinan besar terjadi:
- Kerusakan RAM
- Masalah Overheat (Prosesor)
- Kerusakan Harddisk/SSD
OK